Задание 1.

Данные в таблице clients избыточны. Для начала хотелось бы разделить данную структуру на более конкретные таблицы, например:

- Персональная информация, где отражались бы поля с именем, фамилией, возрастом и городом пользователя;
- Регистрационные данные, где отражалась бы такая информация как логин и дата регистрации.

Затем хотелось бы изучить данные о поле city_id и понять откуда берутся данные идентификатора города. Правильно было бы иметь отдельную таблицу, которая бы содержала id города и прочую необходимую информацию о нем, в том числе и название города. Если такой таблицы нет, то данное поле не представляет никакой полезной информации о клиенте. Если же подобная таблица имеется, то поле city_id должно быть помечено как внешний ключ.

Похожая ситуация и с полем client_id, которое должно быть помечено как первичный ключ. В добавок, если база данных позволяет, то неплохо было бы добавить автоинкремент на данное поле.

Для поля age необходимо было добавить ограничения на возраст клиента, хотя бы на запрет ввода отрицательных значений.

Аналогично и с полем reg_date. Для него хорошо было бы иметь триггер на проверку добавления корректной даты регистрации иначе пользователь может зарегистрироваться в будущем или в момент, когда регистрация еще не работала, то есть в прошлом. Хорошей практикой было бы добавить стандартное значение для этого поле на текущую дату.

Помимо выше перечисленного отсутствуют ограничения на NULL значения, хотя в некоторых полях, таких как client_name, client_surname, login, reg_date.

В рамках системы можно было бы также добавить указание уникальности на поле login, так как данные для авторизации в системе должны быть у каждого пользователя уникальными.

Ну и размер поля login в количестве 30 символов мне кажется может быть недостаточным, поэтому я бы его увеличил бы хотя бы до 40-50 символов.

Задание 2.

Я бы предложил создать таблицу, хранящую исторические данные о стоимости товаров, где для каждого изменения цены была бы прикреплена своя временная метка. Так как эта таблица получилась бы очень огромной, то

можно было бы реализовать секционирование данной таблицы, чтобы получать только данные из необходимых сегментов.

Для составлений различных отчетов для отделов компании можно использовать витрину данных, чтобы в ней хранить только ту информацию, которая будет нужна соответствующему отделу.

Для таблицы items я бы предложил не хранить стоимость товара, а отражать ее только в таблице, где будут фиксироваться изменения цен на товар. В самой таблице оставить только поля item_id и item_name. Данные предложение будет полезно только в случае, если данных об изменении стоимости товара не слишком много. В случае, если записей об изменении будет храниться большое количество, то лучше оставить в таблице items поле item_cost, чтобы исключить join для получения актуальной стоимости. Это будет более производительное решение.

Задание 3.

В условии указано:

```
FROM EMPLOYEE E,
    DEPARTMENTS D
WHERE E.DEPARTMENT_ID = D.DEPARTMENT_ID
```

Здесь происходит присоединение данных из двух таблиц по ключевому полю, а для этого необходимо использовать не запятую, а ключевое слово JOIN, а из условия WHERE убрать: «E.DEPARTMENT_ID = D.DEPARTMENT_ID», так как этому условию должно предшествовать ключевое слово ON.

При указании в SELECT столбца DEPARTMENT_ID может произойти неоднозначное обращение, поэтому нужно точно указать из какой таблицы (DEPARTMENT или EMPLOYEE) брать DEPARTMENT_ID.

В условии WHERE используется агрегатная функция, которая при таком формировании запроса выдаст ошибку о невозможности использования агрегатной функции к конструкции WHERE. Для этого необходимо либо указать: «SUM(SALARY) >= 100000» в блоке HAVING, либо использовать подзапрос.

В конструкцию GROUP BY необходимо добавить столбец DEPARTMENT_ID.

Итоговый запрос с исправлениями должен выглядеть следующим образом:

```sql
SELECT D.DEPARTMENT_ID, DEPARTMENT_NAME, SUM(SALARY) TOTAL_SALARY
FROM EMPLOYEE E JOIN DEPARTMENTS D ON E.DEPARTMENT_ID = D.DEPARTMENT_ID
WHERE
        AND MANAGER_ID = NULL
        AND LOCATION_ID = (SELECT LOCATION_ID FROM LOCATIONS WHERE CITY = 'SEOUL')
GROUP BY D.DEPARTMENT_ID, DEPARTMENT_NAME
HAVING SUM(SALARY) >= 100000;
```

Задание 4.

У меня есть несколько предположений:

1. Карандаш лежит либо рядом со стеной, либо где-то под таким предметом, который невозможно перепрыгнуть, например холодильник;
2. В задаче не сказано где я нахожусь и, возможно, я просто не нахожусь в той же комнате.